UE24CS242B : Operating Systems

Assignment : Orange Problem

Name: Vedaraj H R

SRN:PES1UG24CS643

Section:K

PES-VCS Lab Report:

Phase 1: Object Storage Foundation:-

Screenshot 1A: Output of ./test_objects showing all tests passing**:**

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ nano object.c
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ nano object.c
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_read':
object.c:209:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  209 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure**:**

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Phase 2: Tree Objects:-

Screenshot 2A: Output of ./test_tree showing all tests passing**:**

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format**:**

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ find .pes/objects -type f
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/39/8ee43e2c35a2ea4bfbb0ce89bd8d2a2a168f3a9d6e16f8dd7d9b033c36e5a1
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/7a/a6995ce716851992ffff7ec6d07b93ce60bedf7185f7d9aaa15235b432f75d
.pes/objects/94/cfef4a9c396b1af604a9fabc3e4fae75754ef49b314f28fd83844cccdaa3f9
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/50/da219c4c66a82940f7a8555908504a9bb1ede3773efe8669ad72f1d54966b3
.pes/objects/c1/5daab6ccf47288c923fa5384e07d0e7c63a4f443a1a77c77d51e67b257e822
.pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ xxd. .pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 | head -20
Command 'xxd.' not found, did you mean:
  command 'xxd' from deb xxd (2:9.1.0016-1ubuntu7.11)
Try: sudo apt install <deb name>
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ xxd.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 | head -20
bash: xxd.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39: No such file or directory
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ xxd .pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 | head -20
00000000: 626c 6f62 2036 0048 656c 6c6f 0a         blob 6.Hello.
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Phase 3: The Index (Staging Area):-

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output**:**

```
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ echo "hello" > file1.txt
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ echo "world" > file2.txt
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ ./pes add file1.txt file2.txt
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  README.md
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  commit.c
  untracked:  pes.c
  untracked:  object.c
  untracked:  index.h
  untracked:  tree.h
  untracked:  test_objects
  untracked:  test_tree.c
  untracked:  index.c
  untracked:  Makefile
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  file.txt
  untracked:  .gitignore
  untracked:  tree.c

vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries**:**

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776320825 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776320835 6 file2.txt
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Phase 4: Commits and History:-

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages**:**

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ ./pes log
commit 7aa6995ce716851992ffff7ec6d07b93ce60bedf7185f7d9aaa15235b432f75d
Author: Vedaraj-HR <PES1UG24CS643>
Date:   1776321294

    Add farewellA◆

commit 94cfef4a9c396b1af604a9fabc3e4fae75754ef49b314f28fd83844cccdaa3f9
Author: Vedaraj-HR <PES1UG24CS643>
Date:   1776321264

    Add worldell!

commit 398ee43e2c35a2ea4bfbb0ce89bd8d2a2a168f3a9d6e16f8dd7d9b033c36e5a1
Author: Vedaraj-HR <PES1UG24CS643>
Date:   1776321232

    Initial committial cQ

vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits**:**

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/39/8ee43e2c35a2ea4bfbb0ce89bd8d2a2a168f3a9d6e16f8dd7d9b033c36e5a1
.pes/objects/50/da219c4c66a82940f7a8555908504a9bb1ede3773efe8669ad72f1d54966b3
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7a/a6995ce716851992ffff7ec6d07b93ce60bedf7185f7d9aaa15235b432f75d
.pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
.pes/objects/94/cfef4a9c396b1af604a9fabc3e4fae75754ef49b314f28fd83844cccdaa3f9
.pes/objects/c1/5daab6ccf47288c923fa5384e07d0e7c63a4f443a1a77c77d51e67b257e822
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain**:**

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ cat .pes/refs/heads/main
7aa6995ce716851992ffff7ec6d07b93ce60bedf7185f7d9aaa15235b432f75d
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Final-Full integration test (make test-integration):-

```
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: d6f9d83cc007... Initial commit

Log after first commit:
commit d6f9d83cc00705499b0d5c2e360871e27c7011fd15eafba91f25c4f47cca6976
Author: Vedaraj-HR <PES1UG24CS643>
Date:   1776321823

    Initial commit


--- Second Commit ---
Committed: 951006b44f73... Update file.txt

--- Third Commit ---
Committed: 698101f4552a... Add farewell

--- Full History ---
commit 698101f4552a5922de3411d9e3e3242dc09c77fd3b4433a67c273faad6bb0d70
```

```
--- Third Commit ---
Committed: 698101f4552a... Add farewell

--- Full History ---
commit 698101f4552a5922de3411d9e3e3242dc09c77fd3b4433a67c273faad6bb0d70
Author: Vedaraj-HR <PES1UG24CS643>
Date:   1776321823

    Add farewellA◆

commit 951006b44f73432ccee3a45f055391bd7ae389419c83a74edd0f5673c5bbcf0e
Author: Vedaraj-HR <PES1UG24CS643>
Date:   1776321823

    Update file.txt

commit d6f9d83cc00705499b0d5c2e360871e27c7011fd15eafba91f25c4f47cca6976
Author: Vedaraj-HR <PES1UG24CS643>
Date:   1776321823

    Initial committial cQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
698101f4552a5922de3411d9e3e3242dc09c77fd3b4433a67c273faad6bb0d70

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/69/8101f4552a5922de3411d9e3e3242dc09c77fd3b4433a67c273faad6bb0d70
.pes/objects/84/c5fed72c2dd9a3130e2e9b98033827f62793fb80d0295ccedf5fc7972d96eb
.pes/objects/95/1006b44f73432ccee3a45f055391bd7ae389419c83a74edd0f5673c5bbcf0e
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d6/f9d83cc00705499b0d5c2e360871e27c7011fd15eafba91f25c4f47cca6976
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/de/34fb483a5c16ce29ec3195dd505dfda7e35cc5fe16a99da4b2455b332a93cf
.pes/objects/ec/0af37b159ae92a5850ceaa6e497860729d6158aab2c6833b6068a1e3c5e8ef

=== All integration tests completed ===
vedaraj@Ubuntu:~/PES1UG24CS643-pes-vcs$
```

Phase 5 & 6: Analysis-Only Questions:-

Q5.1)

To implement **pes checkout <branch>:**

Files that need to change in .pes/:

1. **.pes/HEAD** — must be updated to point to the new branch**: ref: refs/heads/<branch>**
2. The working directory files must be updated to match the target branch's tree

Steps:

1. Read the target branch file (**.pes/refs/heads/<branch>**) to get its commit hash
2. Read the commit object to get its tree hash
3. Recursively walk the tree and restore every file to the version stored in that tree
4. Update **.pes/HEAD** to say **ref: refs/heads/<branch>**

What makes it complex:

- You must detect and refuse to overwrite files that have uncommitted changes (dirty files)
- You must handle files that exist in the current branch but not in the target (delete them)
- You must handle files that exist in the target but not in the current branch (create them)
- Sub-directories must be created or deleted as needed
- The index must be updated to reflect the new working directory state

Q5.2)

For each tracked file (in the index), compare three versions:

1. **Working directory version** — compute its SHA-256 hash by reading the actual file
2. **Staged version** — the hash stored in the index
3. **HEAD version** — the hash stored in the HEAD commit's tree

A file is "dirty" (has uncommitted changes) if the working directory hash differs from the index hash (meaning it was modified after staging). If the file also differs between the two branches being switched, checkout must refuse.

The algorithm:

1. Load the current index
2. For each index entry, **stat()** the file and compare **mtime** and **size** to the index — if they differ, the file is potentially modified
3. If potentially modified, recompute its blob hash and compare to the index entry's hash — if they differ, the file is dirty
4. If the dirty file's blob hash differs from the target branch's version of that file, refuse checkout

Q5.3)

Detached HEAD means **.pes/HEAD** contains a commit hash directly instead of **ref: refs/heads/main**. This happens when you checkout a specific commit hash rather than a branch name.

What happens if you commit in this state: New commits are created and chained correctly, but no branch pointer is updated. The commits exist in the object store, but since no branch file contains their hash, they have no stable reference. If

you check out a branch, these commits become "dangling" — reachable only if you remember their hash.

Recovery:

1. Note the commit hash from the output of the last **commit** command (or from the detached HEAD file)
2. Create a new branch pointing to that hash: this is done by writing the hash into a new file at **.pes/refs/heads/recovery-branch**
3. Then checkout that branch normally

Q6.1)

Algorithm (Mark and Sweep):

1. **Mark phase:** Starting from all branch refs and HEAD, perform a graph traversal:
    - Start with all commit hashes in **.pes/refs/heads/***
    - For each commit: add its hash to a "reachable" set, then add its tree hash
    - For each tree: recursively add all blob hashes and subtree hashes
    - Follow parent commit pointers until the root
2. **Sweep phase:** Walk all files in **.pes/objects/**. For each object file, derive its hash from the path. If the hash is not in the reachable set, delete the file.

Data structure: A hash set (implemented as a hash table or sorted array of 64-char hex strings). Lookup is O(1) average.

Estimate for 100,000 commits, 50 branches:

- 100,000 commits (one object each)
- Each commit references a tree; assume ~10 tree objects per commit → ~1,000,000 tree objects (but many are shared, so maybe ~500,000 unique)
- Each tree references blobs; assume ~50 files per tree → potentially 5,000,000 blob references, but with deduplication maybe ~200,000 unique blobs
- Total objects to visit: roughly **800,000 to 2,000,000**

Q6.2)

The race condition:

1. Thread A (commit operation): Creates a new blob object for **file.txt**. The blob is now in the object store, but the commit object (which references it) has not been written yet.
2. Thread B (garbage collector): Scans the object store. It sees the new blob but finds no commit or tree that references it. It marks it as "unreachable" and deletes it.
3. Thread A (commit operation): Now tries to write the commit object referencing the (now deleted) blob. The repository is corrupted.

How Git's real GC avoids this:

- Git's **git gc** uses a "grace period" — objects newer than 2 weeks are never deleted, regardless of reachability. This gives in-progress operations time to complete.

- Git also uses lock files and writes objects to a "loose" area first. A **COMMIT_EDITMSG** lock and ref locks prevent concurrent operations from interleaving.
- The GC first writes a **packed-refs** file atomically before deleting loose objects, ensuring consistency.